

GPU platform slice: executive summary

Platform Engineer take-home · private repo antigenius0910/gpu-platform-slice · 29 September 2026

What I built

A runnable self-service GPU platform on a local k3d cluster. Onboarding a team is a five-line file in Git. One Helm chart turns it into a namespace, RBAC, a GPU quota, a priority tier, and network isolation. Two labelled node pools, datacenter and cloud, advertise 4 fake GPUs each. `make create`, `make verify`, and `make destroy` run on any machine with Docker and k3d. The same `verify.sh` runs in CI on every change to the slice, and every assertion passes.

```
research      gpuCap 4   tier low   pool any
release-evals gpuCap 6   tier high  pool any
inference-eval gpuCap 4   tier high  pool any
caps 4 + 6 + 4 = 14 GPUs > 8 GPUs in the fleet
```

What the verification proves, as the team's own identity

- A request over the cap is refused at admission, and the pod never exists.
- A request inside the cap can still wait: *Insufficient nvidia.com/gpu*. A cap is a ceiling, not capacity set aside.
- The higher tier preempts the lower one when GPUs run out, shown by a *Preempted* event, not a pod status.
- Team A cannot reach team B's service (curl exit 7), with a same-namespace probe getting HTTP 200 before and after.
- A team that may use any pool spills to cloud when the datacenter is full. A team pinned to the datacenter waits instead.
- A team cannot raise its quota, edit its namespace, mint a tier, borrow a higher tier, or run a privileged pod.
- Each case was broken on purpose and seen failing for the right reason. Each runs alone, in any order.

Decisions and why

- **Entitlement lives in Git, in abstract terms.** A spec says *tier: low*, never a PriorityClass name, so the backend can change without touching any team's file. CODEOWNERS routes every change to the platform team.
- **Enforcement happens at admission, not by convention.** Writing the design down first exposed a gap: blocking *create priorityclass* stops a team from minting a tier but not from naming an existing one. One built-in ValidatingAdmissionPolicy now closes it for every team, with no webhook to run.
- **A chart, not a controller.** At three teams, a Helm chart with a JSON schema is enough, and it can be extended live. A controller earns its place at around fifty teams.
- **k3s, pinned to v1.35.8.** It enforces standard NetworkPolicy out of the box, and the k3d default is four minor versions behind kubectl.
- **The suite proves which cluster it is on before it asserts anything.** On a test host, *kubectf* pointed at a different, live cluster. The scripts pin their kubeconfig and check the API server's identity first.
- **What the design makes worse:** caps are oversubscribed, so a low-tier team inside its cap cannot predict when it will run. The exchange is GPU utilization, and the contract states it plainly.

What I dropped, on purpose

Device plugin, GPU Operator, MIG, and time-slicing, which need real GPUs. Terraform, EKS, Karpenter, and Cluster API, which need real accounts and hardware. A second cluster, cross-cluster overflow, and ClusterMesh. Egress policy, fair share, and job queueing. Each one is written up as design in docs/. Node loss was measured on this slice rather than added to the suite. It found a trap: pods on a dead node keep holding their team's quota, so a team at its cap cannot restart its work elsewhere until the platform marks the node out-of-service.

Where to start reading

README.md for the commands and the map, then docs/ARCHITECTURE.md for the diagram, trust assumptions, and trade-offs, and docs/CONTRACT.md for the platform contract. docs/PREDICTIONS.md records each expected result before it ran, including the predictions I got wrong. docs/NOTES.md covers AI use and the defects it introduced.